

Form Builder — Frontend Take-Home Assignment

Time estimate: Most candidates spend 6–8 hours. There is no hard deadline on hours — submit when you're satisfied with the quality of the core features.

What You're Building

A browser-based form builder — think Google Forms, but with the visual quality and interaction patterns of a modern SaaS product.

The application has two modes:

1. **Builder Mode** — Design a form template by adding, configuring, and ordering fields
2. **Fill Mode** — Create a new form instance from a template, fill it out, and export it as a PDF

All data is stored in `localStorage`. No backend, no server, no database required.

The 10 Field Types

Implement the following field types. Each must be fully configurable in Builder Mode and correctly rendered in Fill Mode.

1. Single Line Text

A single-line text input.

Config options:

- Label (*required*)
- Placeholder text
- Required toggle
- Min / max character length
- Prefix text — static text rendered **before** the input (e.g. `https://`)
- Suffix text — static text rendered **after** the input (e.g. `.com`)

2. Multi-line Text

A multi-row textarea.

Config options:

- Label (*required*)
 - Placeholder text
 - Required toggle
 - Min / max character length
 - Number of visible rows
-

3. Number

A numeric input.

Config options:

- Label (*required*)
 - Required toggle
 - Min value / max value
 - Decimal places (0–4)
 - Prefix (e.g. \$)
 - Suffix (e.g. kg)
-

4. Date

A date picker.

Config options:

- Label (*required*)
 - Required toggle
 - **Pre-fill with today's date** — a toggle that, when enabled, automatically sets the field to today's date when a new form instance is opened
 - Min date / max date
-

5. Single Select

Let the user pick one option from a list.

Config options:

- Label (*required*)
- Required toggle
- Options list (add, remove, reorder)
- **Display type** — the same field and options must render in three different ways:
 - **Radio** — stacked list of labelled radio inputs
 - **Dropdown** — a select input that opens a list
 - **Tiles** — horizontally laid out clickable option cards

All three display types must be implemented. The selected option and required validation must behave identically across all three.

6. Multi Select

Let the user pick multiple options from a list.

Config options:

- Label (*required*)
 - Required toggle
 - Options list
 - Min selections / max selections
-

7. File Upload

Let the user attach one or more files.

Config options:

- Label (*required*)
- Required toggle
- Allowed file types (e.g. `.pdf`, `.jpg`, `.png` — comma-separated)
- Max number of files

Note: Files are stored as metadata only (filename, size, type) — no upload to a server. The PDF export should handle the fact that file contents cannot be embedded.

8. Section Header

A non-input display element for visually grouping and labelling sections of a form. It does not capture any value from the user.

Config options:

- Label text
 - Size: **XS** / **Small** / **Medium** / **Large** / **XL** — each size maps to a different visual weight/heading level
-

9. Conditional Logic

Any field can have one or more conditions that control its **visibility** or **required state** based on the value of another field.

Each condition is defined by:

- **Target field** — the field whose value is evaluated
- **Operator** — depends on the target field's type:

Target Field Type	Available Operators
Single Line Text, Multi-line Text	<code>equals</code> , <code>does not equal</code> , <code>contains</code>
Number	<code>equals</code> , <code>is greater than</code> , <code>is less than</code> , <code>is within range</code>
Single Select	<code>equals</code> , <code>does not equal</code>
Multi Select	<code>contains any of</code> , <code>contains all of</code> , <code>contains none of</code>
Date	<code>equals</code> , <code>is before</code> , <code>is after</code>

- **Value** — the value to compare against
- **Effect** — one of:
 - Show this field
 - Hide this field
 - Mark as required

- Mark as not required

Each field also has a **default state** (when no condition is active or when the target field has no value):

- Default visibility: visible or hidden
- Default required: required or not required

Rules to enforce:

- A **hidden field must never be validated** as required, even if it is marked required
 - A **hidden field's value must not appear** in the submitted data or in the PDF export
 - A field **cannot set a condition on itself**
 - If a field has multiple conditions, document your AND / OR decision in the README
-

10. Calculation

A **read-only** field whose value is automatically computed from other Number fields in the form.

Config options:

- Label (*required*)
- Source fields — one or more Number fields in this form
- Aggregation type: **Sum**, **Average**, **Minimum**, **Maximum**
- Decimal places for the result

Behaviour:

- The calculated value must update **in real-time** as the user fills in the source Number fields
 - The field is always read-only in Fill Mode — the user cannot type into it
 - A Calculation field may not use another Calculation field as a source
-

Application Structure

Templates List (Home)

- Shows all saved form templates
- **"New Template"** button — opens Builder Mode with a blank form
- Each template card shows: title, number of fields, number of filled instances, last modified date

- Click a template — opens it in Builder Mode (edit)
- **"New Response"** button on each template card — creates a fresh instance and opens it in Fill Mode

Builder Mode

- **Left panel** — available field types; click or drag to add to the canvas
- **Center canvas** — the ordered list of fields in the form; support reordering (drag-and-drop preferred; up/down buttons are acceptable)
- **Right panel** — configuration panel for the currently selected field
- **Save button** — persists the template to `localStorage`
- **Preview button** — opens Fill Mode inline or in a modal so you can test the form as a builder

Fill Mode

- Renders all fields according to their config
- Conditional fields show and hide in real-time as the user fills values
- Calculation fields update in real-time
- Required fields are validated on submit — hidden required fields are not validated
- **Submit** — saves the filled instance to `localStorage`
- **Download PDF** — exports the filled form (field labels + values) as a PDF

Filled Instances List

- Accessible from a template's detail view or as a separate section
 - Shows all submitted instances for a given template
 - Each entry: submission timestamp, and a **Re-download PDF** button
-

PDF Export

The PDF export must use **browser-native APIs only**. Do not use any third-party PDF generation libraries.

The exported PDF must include:

- The form title
- All visible field labels and their submitted values
- Submission timestamp
- Fields should appear in form order

Hidden fields (due to conditional logic) must not appear in the export.

Technical Requirements

- **React + TypeScript** — strong typing is expected; avoid `any`
 - **No backend** — everything in `localStorage`
 - **No third-party PDF libraries** — browser-native APIs only
 - **No specific component library required** — bring your own styling (Tailwind, CSS Modules, plain CSS — your choice)
 - The app must work correctly after a full page refresh
 - No specific state management library is required — use what you judge is appropriate
-

What to Submit

1. A **GitHub repository** (public or shared with us) with:

- Full source code
- A `README.md` covering:
 - How to run the project locally
 - Your `localStorage` schema and the reasoning behind it
 - Key architectural decisions and trade-offs (especially around conditional logic and component structure)
 - What you would improve with more time

2. An **AI usage log** — a `.md` file in the repository covering:

- Every significant prompt you sent to an AI tool
- What you verified before using the AI output
- Any AI output you rejected or significantly changed — and why
- At least one example where the AI produced something plausible but incorrect

We are not looking for a long log — quality over volume. 5 well-described prompts beat 20 one-liners.

Evaluation Criteria

We are looking for:

- **Product thinking** — Did you make sensible decisions for the 10–20% of cases the spec doesn't cover? Did you document them?
- **Component design** — Can someone add an 11th field type without editing 6 existing files?
- **Conditional logic correctness** — Does it handle chained conditions, hidden-but-required fields, and real-time updates correctly?
- **Type safety** — Do your TypeScript types communicate the design, or are they decorative?
- **PDF export quality** — Does the output look like a real export, or a debug dump?

We are **not** looking for:

- Pixel-perfect design
- 100% feature completeness
- A production-ready, deployable application

Correctness and judgment over completeness.

Questions

For anything genuinely ambiguous and not covered by this spec — do not ask us. Make a decision, document it in your README, and explain your reasoning. That decision-making process is part of what we're evaluating.